

Java Collections: Set and Map - Cheat Sheet

Set is about uniqueness. Map is about key-value lookup. Most Set implementations are thin wrappers around a `Map`, so once you know one family well, the other clicks fast.

The Set interface

A `Set` holds distinct elements. No duplicates allowed. How “distinct” is decided depends on the implementation. Hash-based sets (`HashSet`, `LinkedHashSet`) use `equals` and `hashCode`. Sorted sets (`TreeSet`) use `compareTo` or a supplied `Comparator`. `IdentityHashMap`-backed sets would use `==`, but there’s no standard `IdentityHashSet` (you’d wrap one with `Collections.newSetFromMap`).

Core methods:

```
Set<String> s = new HashSet<>();
s.add("apple");           // true if added, false if already present
s.remove("apple");        // true if removed
s.contains("apple");      // true if present
s.size();
s.isEmpty();
```

Set algebra (all are in-place on the receiver):

```
Set<Integer> a = new HashSet<>(Set.of(1, 2, 3));
Set<Integer> b = new HashSet<>(Set.of(2, 3, 4));

a.addAll(b);             // union:      a = {1, 2, 3, 4}
a.retainAll(b);          // intersection: a = {2, 3, 4} (after reset)
a.removeAll(b);          // difference:  a = {1}      (after reset)
```

Immutable sets (Java 9+):

```
Set<String> fixed = Set.of("a", "b", "c");           // throws on duplicate or null
Set<String> copy  = Set.copyOf(someExistingSet);     // defensive copy
```

`Set.of` rejects nulls and duplicates at construction time. The returned set throws `UnsupportedOperationException` on any mutation.

HashSet

The default `Set` implementation. Backed by a `HashMap` where every value is a dummy sentinel (`PRESENT`, a private static `Object`). It gives $O(1)$ average for `add`, `remove`, and `contains`.

assuming a sane `hashCode`. There are no ordering guarantees, and iteration order can change between JVM runs. It allows one null element.

```
Set<String> seen = new HashSet<>();
seen.add("alice");
seen.add("bob");
seen.add("alice"); // ignored, returns false
System.out.println(seen.size()); // 2
```

If your element type doesn't override `equals` and `hashCode` correctly, `HashSet` will look broken. See [collections-internals.pdf](#) for the contract.

LinkedHashSet

`HashSet` plus a doubly-linked list threading through all entries in insertion order. Iteration order is predictable (insertion order). It uses slightly more memory and has a small iteration overhead vs `HashSet`, but the main operations are still $O(1)$ average.

Use it when you want set semantics but the order in which elements went in still matters (e.g., deduplicating a stream while preserving the original sequence).

```
Set<String> ordered = new LinkedHashSet<>();
ordered.add("c");
ordered.add("a");
ordered.add("b");
// iterates: c, a, b
```

TreeSet

A sorted set backed by a red-black tree. Implements `NavigableSet`. Operations are $O(\log n)$ for `add`, `remove`, and `contains`. Elements must be `Comparable`, or you pass a `Comparator` to the constructor. No nulls, because natural ordering can't compare to null.

Useful navigation methods:

```
TreeSet<Integer> t = new TreeSet<>(Set.of(10, 20, 30, 40, 50));
t.first();           // 10
t.last();            // 50
t.ceiling(25);       // 30 (smallest >= 25)
t.floor(25);         // 20 (largest <= 25)
t.higher(30);        // 40 (strictly >)
t.lower(30);         // 20 (strictly <)
t.headSet(30);       // [10, 20]      elements < 30
t.tailSet(30);       // [30, 40, 50]   elements >= 30
t.subSet(20, 40);    // [20, 30]      [from, to)
```

Reach for `TreeSet` when you need range queries or sorted iteration. For unique sorted IDs, top-N lookups, or “find the closest value”, it’s hard to beat.

EnumSet

A specialized set for enum keys. Backed by a bit vector (a `long` for enums with ≤ 64 values, a `long[]` for larger ones). It’s extremely fast (operations are bit ops) with a tiny memory footprint.

Always reach for this when your set’s elements are an enum type. Don’t use `HashSet<MyEnum>`.

```
enum Color { RED, GREEN, BLUE, YELLOW }

EnumSet<Color> primary = EnumSet.of(Color.RED, Color.GREEN, Color.BLUE);
EnumSet<Color> all     = EnumSet.allOf(Color.class);
EnumSet<Color> none    = EnumSet.noneOf(Color.class);
EnumSet<Color> range   = EnumSet.range(Color.GREEN, Color.YELLOW);
EnumSet<Color> minus   = EnumSet.complementOf(primary); // {YELLOW}
```

`EnumSet` doesn’t allow nulls and isn’t thread-safe (wrap with `Collections.synchronizedSet` if you need that).

CopyOnWriteArraySet

A thread-safe set backed by `CopyOnWriteArrayList`. Every mutation creates a fresh copy of the underlying array. Reads are lock-free and cheap, writes are expensive (full array copy). Good for small, read-mostly sets like a listener registry.

Detailed coverage and trade-offs live in [collections-concurrent.pdf](#).

Set time complexity

Implementation	add	remove	contains	iteration order	allows null
<code>HashSet</code>	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	none (hash)	one null
<code>LinkedHashSet</code>	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	insertion	one null
<code>TreeSet</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$	sorted	no
<code>EnumSet</code>	$O(1)$	$O(1)$	$O(1)$	enum declaration	no
<code>CopyOnWriteArraySet</code>	$O(n)$	$O(n)$	$O(n)$	insertion	yes

Worst case for hash-based sets is $O(n)$ under bad hashing. Since Java 8, `HashMap` (and therefore `HashSet`) treeifies long collision chains, bringing worst case down to $O(\log n)$. Details in

The Map interface

A `Map` holds key-value pairs with unique keys. Values can be anything, including null in most implementations.

Core methods you'll use constantly:

```
Map<String, Integer> m = new HashMap<>();
m.put("apple", 1);
m.get("apple");           // 1
m.getOrDefault("pear", 0); // 0, no NullPointerException
m.containsKey("apple");
m.containsValue(1);
m.remove("apple");
m.size();
m.keySet(); // Set<K> view
m.values(); // Collection<V> view
m.entrySet(); // Set<Map.Entry<K,V>> view
```

The Java 8 additions are where modern `Map` code really lives:

```
m.putIfAbsent("apple", 1);
m.computeIfAbsent("apple", k -> expensiveLoad(k));
m.computeIfPresent("apple", (k, v) -> v + 1);
m.compute("apple", (k, v) -> v == null ? 1 : v + 1);
m.merge("apple", 1, Integer::sum);
m.replace("apple", 5);
m.forEach((k, v) -> System.out.println(k + " -> " + v));
```

Immutable maps (Java 9+):

```
Map<String, Integer> small = Map.of("a", 1, "b", 2); // up to 10 pairs
Map<String, Integer> big   = Map.ofEntries(
    Map.entry("a", 1),
    Map.entry("b", 2),
    Map.entry("c", 3)
);
Map<String, Integer> copy = Map.copyOf(someExistingMap);
```

These reject null keys and null values.

HashMap

The default map. Hash table with separate chaining (linked lists that turn into red-black trees once a bucket gets long enough). It gives $O(1)$ average for `get`, `put`, and `remove`. One null key allowed,

multiple null values allowed. No ordering guarantees. It's not thread-safe, and concurrent modification can corrupt the structure (use `ConcurrentHashMap` instead).

```
Map<String, Integer> counts = new HashMap<>();
String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};

// Word count, idiomatic
for (String w : words) {
    counts.merge(w, 1, Integer::sum);
}
// {apple=3, banana=2, cherry=1}

// Grouping into lists
Map<Integer, List<String>> byLength = new HashMap<>();
for (String w : words) {
    byLength.computeIfAbsent(w.length(), k -> new ArrayList<>()).add(w);
}
```

For how `HashMap` actually works under the hood (load factor, treeification threshold, resize behavior), see [collections-internals.pdf](#).

LinkedHashMap

`HashMap` with a doubly-linked list threading entries in insertion order. Two modes:

Insertion order (default). Entries iterate in the order they were first inserted.

Access order (constructor flag). Entries move to the end of the list on every `get` or `put`. This is the building block for an LRU cache.

Classic LRU cache in about ten lines:

```

class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LruCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder = true
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

LruCache<String, String> cache = new LruCache<>(3);
cache.put("a", "1");
cache.put("b", "2");
cache.put("c", "3");
cache.get("a"); // a is now most recently used
cache.put("d", "4"); // evicts b (least recently used)

```

TreeMap

Sorted map backed by a red-black tree. Implements `NavigableMap`. Operations are $O(\log n)$ for `get`, `put`, and `remove`. Keys must be `Comparable` or you supply a `Comparator`. No null keys (same reason as `TreeSet`). Null values are fine.

```

TreeMap<Integer, String> tm = new TreeMap<>();
tm.put(10, "ten");
tm.put(20, "twenty");
tm.put(30, "thirty");

tm.firstKey(); // 10
tm.lastKey(); // 30
tm.ceilingKey(15); // 20
tm.floorKey(15); // 10
tm.higherKey(20); // 30
tm.lowerKey(20); // 10
tm.headMap(20); // {10=ten} keys < 20
tm.tailMap(20); // {20=twenty, 30=...} keys >= 20
tm.subMap(10, 30); // {10=ten, 20=twenty} [from, to)
tm.pollFirstEntry(); // removes and returns smallest
tm.pollLastEntry(); // removes and returns largest

```

Reach for `TreeMap` when you need range scans, floor/ceiling lookups, or sorted iteration by key.

EnumMap

Specialized map for enum keys. Backed by a plain array indexed by the enum's ordinal. Very fast (array indexing, no hashing) with a compact memory footprint. Always pick this over

`HashMap<MyEnum, V>` .

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumMap<Day, Integer> hoursWorked = new EnumMap<>(Day.class);
hoursWorked.put(Day.MON, 8);
hoursWorked.put(Day.TUE, 9);
hoursWorked.put(Day.WED, 7);
// Iteration order is the enum declaration order: MON, TUE, WED, ...
```

No null keys. Null values are allowed.

WeakHashMap

Keys are held via weak references. When no strong reference to a key exists anywhere else, the entry becomes eligible for garbage collection. Niche but useful for caches keyed by class loaders, per-instance metadata, or listener registries.

Watch out: if a value holds a strong reference back to its key, the key stays alive forever. The whole point breaks.

```
WeakHashMap<Object, String> wm = new WeakHashMap<>();
Object key = new Object();
wm.put(key, "data");

key = null; // last strong ref dropped
System.gc(); // wm may now be empty (eventually)
```

`WeakHashMap` isn't thread-safe and has the same iteration semantics as `HashMap` .

IdentityHashMap

Compares keys with `==` instead of `equals` . Uses `System.identityHashCode` instead of the object's `hashCode` . Two distinct `String` objects with the same characters are different keys here. It's used in topology tracking (e.g., graph traversal where you need to know “have I visited this exact object”), serialization frameworks, and identity-based caches.

```
IdentityHashMap<String, Integer> im = new IdentityHashMap<>();
String a = new String("hello");
String b = new String("hello");
im.put(a, 1);
im.put(b, 2);
System.out.println(im.size()); // 2, because a != b by reference
```

Don't reach for this unless you specifically need identity semantics. It will surprise teammates.

Hashtable

Legacy class from Java 1.0. Synchronized on every method. Don't use in new code. For thread-safe maps, use `ConcurrentHashMap` (covered in `collections-concurrent.pdf`). It doesn't allow null keys or null values. It's still around because of `Properties`, which extends it.

Properties

`Properties` extends `Hashtable<Object, Object>` but is meant to hold `String` to `String`. It loads and saves `.properties` files via `load(InputStream)` and `store(OutputStream, comment)`, and is used heavily for app config in older codebases (and still in some modern ones).

```
Properties props = new Properties();
try (InputStream in = Files.newInputStream(Paths.get("app.properties"))) {
    props.load(in);
}
String dbUrl = props.getProperty("db.url", "jdbc:default");
```

Legacy but ubiquitous. Worth knowing for interview questions and reading old code.

Map time complexity

Implementation	put	get	remove	null key	ordering	thread-safe
HashMap	O(1) avg	O(1) avg	O(1) avg	one	none	no
LinkedHashMap	O(1) avg	O(1) avg	O(1) avg	one	insertion/access	no
TreeMap	O(log n)	O(log n)	O(log n)	no	sorted by key	no
EnumMap	O(1)	O(1)	O(1)	no	enum declaration	no
WeakHashMap	O(1) avg	O(1) avg	O(1) avg	yes	none	no
IdentityHashMap	O(1) avg	O(1) avg	O(1) avg	yes	none	no
Hashtable	O(1) avg	O(1) avg	O(1) avg	no	none	yes (slow)

Iteration patterns

The `entrySet()` loop is the right default when you need both keys and values:

```
for (Map.Entry<String, Integer> e : map.entrySet()) {  
    System.out.println(e.getKey() + " -> " + e.getValue());  
}
```

Other variants:

```
for (String k : map.keySet()) { ... } // keys only  
for (Integer v : map.values()) { ... } // values only  
  
map.forEach((k, v) -> System.out.println(k + " -> " + v)); // Java 8+  
  
map.entrySet().stream()  
    .filter(e -> e.getValue() > 10)  
    .forEach(e -> System.out.println(e.getKey()));
```

`keySet()`, `values()`, and `entrySet()` return live views. Modifying them modifies the underlying map:

```
map.keySet().remove("apple"); // removes the "apple" entry from map
map.values().clear();         // clears the whole map
```

Iterating a view and calling `map.put` at the same time throws `ConcurrentModificationException`. Use `Iterator.remove()` for safe deletion during iteration, or collect keys first and mutate after.

Modern map idioms

These four methods replace most of the verbose patterns you'd write pre-Java-8.

`getOrDefault` instead of null-checking:

```
// Old:
Integer count = map.get(key);
if (count == null) count = 0;

// New:
int count = map.getOrDefault(key, 0);
```

`putIfAbsent` instead of `containsKey` then `put`:

```
// Old:
if (!map.containsKey(key)) map.put(key, value);

// New:
map.putIfAbsent(key, value);
```

`computeIfAbsent` for memoization and lazy initialization:

```
Map<String, List<Order>> ordersByCustomer = new HashMap<>();
for (Order o : orders) {
    ordersByCustomer
        .computeIfAbsent(o.customerId(), k -> new ArrayList<>())
        .add(o);
}
```

`merge` for accumulator patterns:

```
// Word count
Map<String, Integer> counts = new HashMap<>();
for (String word : words) {
    counts.merge(word, 1, Integer::sum);
}

// Sum totals per customer
Map<String, BigDecimal> totals = new HashMap<>();
for (Order o : orders) {
    totals.merge(o.customerId(), o.amount(), BigDecimal::add);
}
```

`compute` for conditional updates that need the current value:

```
// Cap a counter at 100
map.compute(key, (k, v) -> v == null ? 1 : Math.min(v + 1, 100));
```

Immutable maps

Three options, all subtly different:

```
// Java 9+, true immutable, rejects null keys and values
Map<String, Integer> a = Map.of("x", 1, "y", 2);

// Java 9+, for more than 10 entries
Map<String, Integer> b = Map.ofEntries(
    Map.entry("x", 1),
    Map.entry("y", 2),
    Map.entry("z", 3)
);

// Java 10+, defensive copy
Map<String, Integer> c = Map.copyOf(someMap);

// Old approach: unmodifiable VIEW (the backing map can still change!)
Map<String, Integer> d = Collections.unmodifiableMap(someMap);
someMap.put("new", 99); // d now also sees "new"
```

`Collections.unmodifiableMap` is a view, not a copy. If the original map mutates, the “unmodifiable” view sees the change. For real immutability, use `Map.copyOf`.

Set backed by a map

A small but useful fact: most `Set` implementations are thin wrappers around a `Map`.

`HashSet` wraps `HashMap`. `LinkedHashSet` wraps `LinkedHashMap`. `TreeSet` wraps `TreeMap`.

Every set element is stored as a key in the underlying map, with a shared dummy `Object` as the value. That's why the time complexities match exactly and why `Set` rarely surprises you once you understand `Map`.

If you ever need a set backed by a custom map type (say, a thread-safe set backed by `ConcurrentHashMap`), use:

```
Set<String> set = Collections.newSetFromMap(new ConcurrentHashMap<>());
```

Best practices

Pick `HashMap` / `HashSet` by default. Switch to `LinkedHashMap` / `LinkedHashSet` only when iteration order matters.

Use `TreeMap` / `TreeSet` when you need sorted iteration, range queries, or floor/ceiling lookups.

Use `EnumMap` / `EnumSet` whenever your key type is an enum. They're faster and smaller.

Reach for `computeIfAbsent` and `merge` before writing manual `containsKey` checks.

Prefer `Map.of` / `Set.of` for small immutable constants. They're cheap and intent is obvious.

Always override `equals` AND `hashCode` together for keys and elements (see `collections-internals.pdf`).

Use `getOrDefault` to dodge null-pointer landmines.

For thread-safe maps, jump straight to `ConcurrentHashMap`. Skip `Hashtable` and `Collections.synchronizedMap`.

Iterate via `entrySet()` when you need both keys and values. Don't loop over `keySet()` and call `get` inside.

Common gotchas

Mutating a key after putting it in a hash-based collection. The hash bucket is decided at insert time. If you mutate fields that affect `hashCode`, the entry becomes unreachable.

Forgetting to override `hashCode` when you override `equals`. Two "equal" objects with different hashes will end up in different buckets. `contains` will return false.

`Collections.unmodifiableMap` isn't immutable. It's a read-only view over a mutable map. The view changes if the underlying map changes.

`Map.of` and `Set.of` reject null. Drop a null in there and you get `NullPointerException` at construction.

`TreeMap` and `TreeSet` don't allow null keys. They try to call `compareTo(null)`, which blows up.

`HashMap` isn't thread-safe. Concurrent puts can corrupt the bucket structure and even cause infinite loops in older JDKs. Use `ConcurrentHashMap`.

Iterating and modifying simultaneously throws `ConcurrentModificationException`. Use `Iterator.remove()` or collect changes and apply after.

`WeakHashMap` keys disappear silently. Don't iterate expecting a stable view. Don't use it for anything you can't afford to lose.

`IdentityHashMap` treats equal-by-value objects as distinct keys. Don't use it unless you specifically need reference identity.

`Hashtable` and `Properties` are synchronized but slow. Don't use them in performance-sensitive new code.

`LinkedHashMap` access-order mode requires the constructor flag. A default `LinkedHashMap` won't behave like an LRU cache even if you override `removeEldestEntry`.

Quick reference

When you need...	Use
Fast unordered set	<code>HashSet</code>
Set with insertion order	<code>LinkedHashSet</code>
Sorted set, range queries	<code>TreeSet</code>
Set of enum values	<code>EnumSet</code>
Thread-safe read-mostly set	<code>CopyOnWriteArraySet</code>
Fast unordered map	<code>HashMap</code>
Map with insertion or access order	<code>LinkedHashMap</code>
LRU cache	<code>LinkedHashMap</code> (access-order)
Sorted map, floor/ceiling, range scans	<code>TreeMap</code>
Map keyed by enum	<code>EnumMap</code>
Cache keyed by lifetime of a key object	<code>WeakHashMap</code>
Identity-based key comparison	<code>IdentityHashMap</code>
Thread-safe map	<code>ConcurrentHashMap</code> (see concurrent)
<code>.properties</code> file loading	<code>Properties</code>
Small immutable map/set	<code>Map.of</code> / <code>Set.of</code>
Defensive copy as immutable	<code>Map.copyOf</code> / <code>Set.copyOf</code>